Lab 05 - Chi Square

October 25, 2026

1 Machine Learning Pipeline, Cross-Validation & Statistical Testing

CSE-4116 | Lab 5

Implement and compare multiple machine learning algorithms, evaluate model performance using k-fold cross-validation and statistical tests.

[1]:
```
import numpy as np
import pandas as pd

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from scipy.stats import chi2_contingency
```

1.1 1. Load the Dataset

[2]:
```
url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/
 ↪titanic.csv'
df = pd.read_csv(url)

print(df.shape)
display(df.head())
display(df.dtypes)
```

```
(891, 12)
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

```
                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th…  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

```
PassengerId      int64
Survived         int64
Pclass           int64
Name               str
Sex                str
Age            float64
SibSp            int64
Parch            int64
Ticket             str
Fare           float64
Cabin              str
Embarked           str
dtype: object
```

1.1.1 Task 1

[3]:
```
# Keep only the variables needed for this lab.

features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
target = 'Survived'

X = df[features]
y = df[target]

display(X.head())
print('Target distribution:')
print(y.value_counts())
```

```
   Pclass     Sex   Age  SibSp  Parch     Fare Embarked
0       3    male  22.0      1      0   7.2500        S
1       1  female  38.0      1      0  71.2833        C
2       3  female  26.0      0      0   7.9250        S
3       1  female  35.0      1      0  53.1000        S
4       3    male  35.0      0      0   8.0500        S
```

```
Target distribution:
Survived
0    549
1    342
Name: count, dtype: int64
```

1.2 2. Build the Preprocessor

[4]:
```
def build_preprocessor(numerical_features, categorical_features):
    """Create the numerical + categorical preprocessing pipeline."""

    # TODO 1: create numerical preprocessing
    numerical_transformer = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    # TODO 2: create categorical preprocessing
    categorical_transformer = Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('encoder', OneHotEncoder(handle_unknown='ignore'))
    ])

    # TODO 3: combine both using ColumnTransformer
    preprocessor = ColumnTransformer([
        ('numerical', numerical_transformer, numerical_features),
        ('categorical', categorical_transformer, categorical_features)
    ])

    return preprocessor
```

[5]:
```
# Complete these two lists

numerical_features = ['Age', 'SibSp', 'Parch', 'Fare']

categorical_features = ['Pclass', 'Sex', 'Embarked']

preprocessor = build_preprocessor(numerical_features, categorical_features)
preprocessor
```

[5]:
```
ColumnTransformer(transformers=[('numerical',
                                 Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
                                                 ('scaler', StandardScaler())]),
                                 ['Age', 'SibSp', 'Parch', 'Fare']),
                                ('categorical',
                                 Pipeline(steps=[('imputer',
```

```
SimpleImputer(strategy='most_frequent')),
                                                  ('encoder',
OneHotEncoder(handle_unknown='ignore'))]),
                                 ['Pclass', 'Sex', 'Embarked'])])
```

1.3 3. Build the Machine-Learning Pipeline

[6]:
```
def build_pipeline(preprocessor, model):
    """Return a preprocessing + model Pipeline."""

    return Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])
```

1.4 4. Compare Multiple Algorithms

[7]:
```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=200, random_state=42)
}

models
```

[7]:
```
{'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
 'Decision Tree': DecisionTreeClassifier(random_state=42),
 'Random Forest': RandomForestClassifier(n_estimators=200, random_state=42)}
```

1.5 5. 5-Fold Cross-Validation

[8]:
```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

def evaluate_model(model, X, y, preprocessor, cv):
    """Return the five cross-validation accuracy scores."""

    # TODO 1: build the complete pipeline
    pipeline = build_pipeline(preprocessor, model)

    # TODO 2: calculate cross-validation scores
    scores = cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')

    # TODO 3: return scores
    return scores
```

[9]:
```
results = {}
```

```
for name, model in models.items():
    results[name] = evaluate_model(model, X, y, preprocessor, cv)

for name, scores in results.items():
    print(f'{name}: {scores}')
```

```
Logistic Regression: [0.78212291 0.80337079 0.79775281 0.78089888 0.82022472]
Decision Tree: [0.80446927 0.80898876 0.74719101 0.76404494 0.79775281]
Random Forest: [0.83798883 0.79213483 0.78651685 0.82022472 0.8258427 ]
```

1.6 6. Summarize the Results

[10]:
```
summary = []

for name, scores in results.items():
    summary.append({
        'Model': name,
        'Mean Accuracy': scores.mean(),
        'Std Accuracy': scores.std()
    })

results_df = pd.DataFrame(summary).sort_values('Mean Accuracy', ascending=False)
results_df
```

[10]:

	Model	Mean Accuracy	Std Accuracy
2	Random Forest	0.812542	0.019886
0	Logistic Regression	0.796874	0.014567
1	Decision Tree	0.784489	0.024431

1.7 7. Statistical Test: Chi-Square Test of Independence

[11]:
```
# build the contingency table for Sex vs Survived
contingency_table = pd.crosstab(df['Sex'], df['Survived'])

# perform the chi-square test of independence
chi2, p_value, dof, expected = chi2_contingency(contingency_table)

print(contingency_table)
print('Chi-square statistic:', chi2)
print('Degrees of freedom:', dof)
print('p-value:', p_value)

alpha = 0.05
if p_value < alpha:
    print('Reject H0: Sex and survival are statistically associated.')
else:
    print('Do not reject H0: There is insufficient evidence of an association␣
 ↪between sex and survival.')
```

```
Survived    0    1
Sex
female     81  233
male      468  109
Chi-square statistic: 260.71702016732104
Degrees of freedom: 1
p-value: 1.1973570627755645e-58
Reject H0: Sex and survival are statistically associated.
```